FEDERAL POLYTECHNIC, MUBI
ADAMAWA STATE

DEPARTMEN OF COMPUTER SCIENCE

PROGRAM: ND II (COMPUTER SCIENCE)

LECTURE NOTE ON

COM 213 (UNIFIED MODELLING LANGUAGE (UML))

COMPILED BY: M. Z BELLO

Introduction

The Unified Modelling Language (UML) is a graphical language for visualizing, specifying and constructing the artifacts of a software intensive system. The Unified Modelling Language offers a standard way to write a system's blueprints, including conceptual things such as business processes and system functions as well as concrete things such as programming language statements, database schemas, and reusable software components. One of the purposes of UML was to provide the development community with a stable and common design language that could be used to develop and build computer applications. Using UML, IT professionals could now read and disseminate system structure and design plans just as construction workers have been doing for years with blueprints of buildings.

SYSTEM MODELING

System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.

It is an interdisciplinary study that uses models to conceptualize and construct systems in business and IT development. System modeling has now come to mean representing a system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).

IMPORTANCE OF SYSTEM MODELING

Importance of system modeling can be summarized as follows:

i. **Understanding the functionality of the system**: System modeling helps the analyst to understand the functionality of the system, and models that are used to communicate with customers
ii. **Clarifying existing systems**: Models of the existing system are used during requirements engineering to clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system.
iii. **Explaining proposed requirements**: Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.

iv. **Generating system implementation**: In a model-driven engineering process, it is possible to generate a complete or partial system implementation from the system model
v. **Gaining detailed insight**: The use of computer simulations has given us the ability to gain detailed insight into problems that may not have been obvious at the beginning. It also allows us to target these issues and build structures or products which are more fit for the function.
vi. **Analyzing complex systems**: Mathematical modeling is used to analyze the dynamic interactions between several components of a biological system, with the aim to understand the behavior of the system as a whole.
vii. **Evaluating programs**: Systems models could serve to more directly evaluate a program by mapping the relationships within an organization or society that took place over the time of the program.

TYPES OF SYSTEM MODELING

There are many different types of models expressed in a diverse array of modeling languages and tool sets. Here are some of the types of system modeling:

i. **Functional modeling**: is a structured representation of the functions (such as activities, processes, actions, operations) within the modeled system or subject area.
ii. **Architectural modeling**: This type of modeling uses the systems architecture to conceptually model the structure, behavior, and more views of a system. It shows the principal sub-systems and how they are composed of other entities.
iii. **Business Process Modeling**: This is a graphical representation for specifying business processes in a workflow.
iv. **Performance modeling**: This type of modeling focuses on the overall system metrics such as power, performance, and cost. Performance models are created using Resource and Behavior blocks. These analyses are used to identify capacity, limitations and system bottlenecks.
v. **Data processing models**: These models show how data is processed at different stages.
vi. **Composition models:** These models show how entities are composed of other entities.

vii. **Classification models**: These models show how entities have common characteristics.
viii. **Stimulus/response models:** These models show the system's reaction to events.

PRINCIPLE OF MODELING

There are several principles that guide the process of modeling:

i. **Choice of models:** The choice of models to create has a profound influence on how a problem is attacked and how a solution is shaped. The right models will highlight the most critical development problems, while wrong models can mislead and focus on irrelevant issues
ii. **Levels of precision**: Every model can be expressed at different levels of precision, ranging from quick and simple executable models to complex details such as cross-system interfaces or networking issues. The best models allow you to choose your degree of detail, depending on the viewer's needs
iii. **Connection to reality:** The best models are connected to reality, ensuring that the gap between the analysis model and the system's design model is minimal. Failing to bridge this gap can cause the system to diverge over time
iv. **Multiple models**: No single model is sufficient for a non-trivial system. A comprehensive approach involves creating a small set of nearly independent models to better understand and predict the system's behavior
v. **Simplicity:** Models should be simple and easy to understand, as complexity can hinder effective communication and decision-making
vi. **Gradual development**: Models should be developed gradually, allowing for heavy client contact and iterative refinement
vii. **Abstraction**: Models are abstractions of the real system, representing only the relevant features of the system being analyzed
viii. **Transfer of information**: Models should focus on the transfer of information, ensuring that they effectively communicate the desired structure and behavior of the system.

SYSTEM MODEL

System model is the process of developing abstract models of a system, with each model representing a different view or perspective of the system.

TYPES OF SYSTEM MODEL

Some of the key types of system models are:

i. **Data processing model**: showing how the data is processed at different stages
ii. **Composition model**: showing how entities are composed of other entities
iii. **Architectural model**: showing principal sub-systems
iv. **Classification model**: showing how entities have common characteristics
v. **Stimulus/response model**: showing the system's reaction to events
vi. **Behavioral models**: used to describe the overall behavior of a system.
vii. **Flow control model**: used to model queue management, flow control, arbitration, and scheduling in a system
viii. **Performance model**: used to model the overall system metrics such as power, performance, and cost.

SYSTEM MODELING TOOL

Some common types of system modeling tools include:

i. **Function modeling**: Techniques such as Functional Flow Block Diagram and IDEF0 are used to model the functionality of a system and its components
ii. **Architectural modeling**: This approach uses the system architecture to conceptually model the structure, behavior, and various views of a system
iii. **Business Process Modeling Notation (BPMN**): A graphical representation for specifying business processes in a workflow, which can also be considered a system modeling language
iv. **SysML tools:** These are software applications that support some functions of the Systems Modeling Language, which is used for modeling systems in various domains

EXAMPLES OF SYSTEM MODELING TOOL

Here are some examples of system modeling tools:

UML: UML is a graphical modeling language used mainly by software engineers to visualize software systems. It supports a wide range of diagrams, including use case diagrams, class diagrams, and sequence diagrams

SysML Designer: SysML Designer is a graphical tool used to edit and visualize SysML models. It supports the creation of diagrams such as block definition diagrams, activity diagrams, and state machine diagrams

Agilian: Agilian is a software engineering diagramming tool that supports all contemporary modeling notations. It provides a flexible modeling environment for agile software development practitioners to communicate effectively with UML, BPMN, ERD, DFD, and mind maps.

Unified Modeling Language (UML)

UML, or Unified Modeling Language, is a visual modeling language that helps software developers, system architects, and business professionals with modeling, design, and analysis.

Some key points about UML include:

- UML is a standardized modeling language consisting of an integrated set of diagrams
- It is used for specifying, visualizing, constructing, and documenting the artifacts of software systems, as well as for business modeling and other non-software systems
- UML diagrams describe the boundary, structure, and behavior of the system and the objects within it
- UML is flexible and can be used to model different types of applications, running on any type and combination of hardware, operating system, programming language, and network.

ORIGIN OF UML

The Unified Modeling Language (UML) was developed in the mid-1990s with the goal of providing a standard way to visualize the design of a system. It was created by a team of software engineers, including Grady Booch, Ivar Jacobson, and James Rumbaugh, who were working at Rational Software at the time. The development of UML was motivated by the desire to standardize the disparate notational systems and approaches to software design that were being used at the time.

The UML creators each brought their own expertise to the development of the language. Booch's method was flexible and useful for designing and creating objects, Jacobson's method contributed a great way to work on use-cases and high-level design, and Rumbaugh's method was useful for handling complex systems. Additionally, behavioral models and state-charts were added to the UML by David Harel. The UML Partners, a consortium that included companies like HP, DEC, IBM, and Microsoft, was organized in 1996 to complete the UML specification and propose it to the Object Management Group (OMG) for standardization. The UML 1.0 draft was proposed to the OMG by the UML Partners and was eventually adopted as a standard in 1997. In 2005, the International Organization for Standardization (ISO) also approved UML as an ISO standard.

USES OF UML

Here are some of the uses of UML:

i. UML diagrams are most commonly used to analyze existing software, model new software, and plan software development and prioritization
ii. UML diagrams are used to manage processes and projects.
iii. UML diagrams are used to model and design software systems before coding the software application
iv. UML diagrams are used as documentation for the workflow of the project after the code has been written
v. UML diagrams provide a way to visualize many aspects of a project and who is responsible for what activity
vi. UML can also be used to model non-software systems, such as workflow in legal systems, medical electronics and patient healthcare systems, and the design of hardware.

TYPES OF UML DIAGRAMS

There are many types of UML diagrams that help model, these behaviors divided into two main categories: structural diagrams and behavioral diagrams

Structural Diagrams

- **Class Diagram**: Shows the structure of a system by illustrating the classes, attributes, operations, and relationships between objects.

- **Object Diagram**: Shows a snapshot of the instances of the classes in a system and their relationships.
- **Component Diagram**: Shows the components of a system and their dependencies.
- **Deployment Diagram:** Shows the physical deployment of components in a system.

Behavioral Diagrams

- **Use Case Diagram**: Gives a graphic overview of the actors involved in a system, different functions needed by those actors, and how these different functions interact.
- **Activity Diagram**: Shows the flow of activities in a system, including actions, decisions, and branching paths.
- **State Machine Diagram**: Shows the states and transitions of an object or system.
- **Sequence Diagram**: Shows how objects interact with each other and the order those interactions occur.
- **Communication Diagram**: Shows the interactions between objects in a system.
- **Interaction Overview Diagram**: Shows the interactions between objects in a system at a high level.

RELEVANCE OF UML IN SOFTWARE DEVELOPMENT PROCESS

The relevance of UML in the Unified Software Development Process is that it helps project teams communicate, explore potential designs, and validate the architectural design of the software. UML diagrams can be used as a way to visualize a project before it takes place or as documentation for a project afterward. UML helps software engineers, businessmen, and system architects with modeling, design, and analysis. UML is linked with object-oriented design and analysis and makes the use of elements and forms. UML is still relevant today and is used in an Agile environment.

UML SYMBOL SET

UML symbols help define the structural and behavioral aspects of a system, representing things, relationships, and diagrams.

The main building blocks of a UML model are:

- Structural Things: These are the entities represented by specific UML notations and symbols. Some of the commonly used structural things are:
  - ✓ Class: Represents various objects and is used to define their properties and operations.
  - ✓ Object: Represents an instance of a class
  - ✓ Interface: Represents a collection of operations that a class can implement
  - ✓ Collaboration: Represents a group of objects working together to achieve a common goal
  - ✓ Use case: Represents high-level functionalities and how the user will interact with the system
  - ✓ Active class: Represents a class that is currently executing a method or operation
  - ✓ Component: Represents a modular part of a system with well-defined interfaces and responsibilities
  - ✓ Node: Represents a physical component of the system, such as a server or network
- Relationships: These allow you to show how two or more things relate to each other, capturing meaningful connections between elements and describing the functionality of an application. **Some of the commonly used relationships are:**

✓ Association: Represents a relationship between classes, typically named using a verb or verb phrase that reflects the real-world problem domain
✓ Dependency: Represents a relationship where a change in one element may affect another element
✓ Generalization: Represents an inheritance relationship between classes, where one class inherits the properties and operations of another class
✓ Realization: Represents the implementation of an interface by a class
✓ Aggregation: Represents a relationship where one class is composed of one or more other classes, but the parts can exist independently of the whole.
✓ Composition: Represents a relationship where one class is composed of one or more other classes, and the parts cannot exist independently of the whole
✓ Diagrams: UML provides various types of diagrams to represent different aspects of a system, such as class diagrams, use case diagrams, sequence

diagrams, and more. Each diagram has its own set of notations and symbols, which are used to create a visual representation of the system.

VARIOUS TYPES OF UML SOFTWARE

There are several types of UML software available, each with its own features and capabilities. Here is a list of some of the most popular UML software:

i. MagicDraw: MagicDraw is a UML modeling tool that supports a wide range of UML diagrams, including class diagrams, use case diagrams, and activity diagrams. It also supports code generation and reverse engineering.
ii. ArgoUML: ArgoUML is an open-source UML modeling tool that supports a variety of UML diagrams, including class diagrams, use case diagrams, and state diagrams. It also supports code generation and reverse engineering.
iii. Gliffy: Gliffy is a cloud-based diagramming tool that supports a variety of UML diagrams, including class diagrams, sequence diagrams, and activity diagrams. It also integrates with Google Drive, JIRA, and Confluence.
iv. LucidChart: LucidChart is a cloud-based diagramming tool that supports a variety of UML diagrams, including class diagrams, use case diagrams, and activity diagrams. It also supports real-time collaboration and integration with other tools like Google Drive and JIRA.
v. MsVisio: Microsoft Visio is a diagramming tool that supports a variety of UML diagrams, including class diagrams, use case diagrams, and activity diagrams. It also supports integration with other Microsoft tools like Excel and SharePoint.

OBJECT-ORIENTED MODELING (OOM)

OOM is a way of thinking about problems using models organized around real-world concepts, where the fundamental construct is the object, which combines both data structure and behavior. The purpose of OOM is to apply object-oriented concepts to all stages of the software development life cycle. Object-oriented modeling is typically done via use cases and abstract definitions of the most important objects. The most common language used to do object-oriented modeling is the Object Management Group's Unified Modeling Language (UML).

There are three types of models in object-oriented modeling and design:

i. Class Model

ii. State Model
iii. Interaction Model

The reasons to model a system before writing the code are communication, visualization, reduction of complexity, and testing a physical entity before building it.

OBJECT-ORIENTED ANALYSIS AND DESIGN

Object-oriented analysis and design (OOAD) is a software engineering methodology that involves using object-oriented concepts to design and implement software systems. It is a technical approach for analyzing and designing an application, system, or business by applying object-oriented concepts. OOAD involves a number of techniques and practices, including object-oriented programming, design patterns, UML diagrams, and use cases.

BENEFITS OF OBJECTS-ORIENTED MODELING

The benefits of object-oriented modeling include:

i. Faster development of software
ii. Easier maintenance
iii. Effective problem-solving
iv. Better structured programs
v. Higher code reuse

OBJECT-ORIENTED SYSTEM CONCEPTS

Object-oriented system concepts are a set of principles and practices used in software engineering to design and implement software systems. These concepts include:

- Object: An object is an instance of a class that encapsulates data and behavior.
- Class: A class is a blueprint for creating objects that defines the attributes and methods of the objects.
- Polymorphism: Polymorphism is the ability of objects of different classes to be treated as if they were objects of the same class.
- Component: A component is a modular, self-contained unit of software that can be reused in different contexts
- Abstraction: Abstraction is the process of identifying the essential characteristics of an object and ignoring the irrelevant details.

- Interface: An interface is a set of methods that define the behavior of a class. It specifies what a class can do, but not how it does it.
- Inheritance: Inheritance is the process of creating a new class from an existing class by inheriting its attributes and methods. This allows for code reuse and promotes a hierarchical organization of classes.

STATE OF AN OBJECT

The state of an object refers to the condition of the object at a particular time period. It is a situation occurring for a finite time period in the lifetime of an object, in which it fulfills certain conditions, performs certain activities, or waits for certain events to occur.

- Events: Events are occurrences that may trigger a state transition in an object. Event types include an explicit signal from outside the system, an invocation from inside the system, the passage of a designated period of time, or a designated condition becoming true.
- Transactions: A transaction is a sequence of operations that are treated as a single unit of work.
- Messages: Messages are objects that describe something that has occurred in the application. They can be table.

CONCEPTUAL DIAGRAMS

A conceptual diagram is a visual representation of the ways in which abstract concepts are related. It is used as an aid in visualizing processes or systems at a high level through a series of unique lines and shapes.

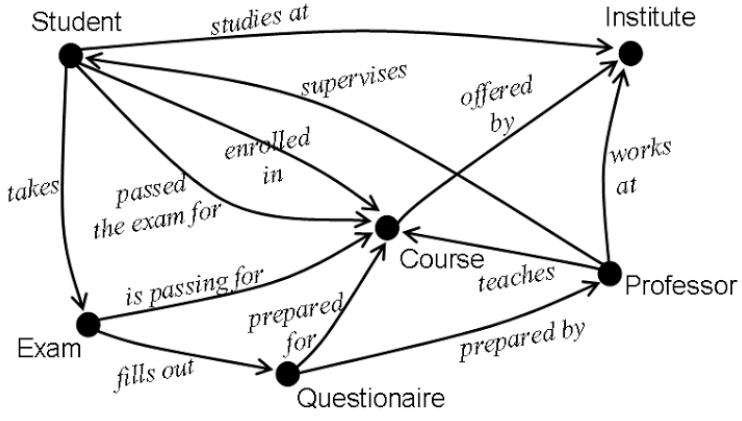

Fig 1. The conceptual diagram

There are two main types of conceptual diagrams:

i. Causal/associative models
ii. Descriptive/Structural diagrams

Causal/Associative Models generally, provide predictions that can be tested and falsified, whereas **descriptive/structural diagrams** provide paradigmatic ways of thinking through phenomena.

CLASS DIAGRAM

A class diagram is a type of static structure diagram in the Unified Modeling Language (UML) that describes the structure of a system by showing the system classes, their attributes, operations (or methods), and the relationships among objects.

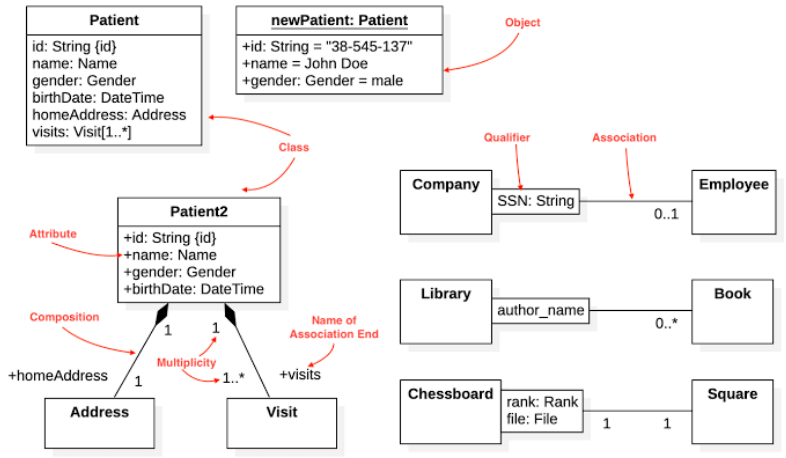

Fig 2. The Class Diagram

USES OF CLASS DIAGRAM

Here are some uses of class diagrams:

i. **Modeling the system**: Class diagrams can be used to model the objects that make up the system and to display the relationships between the objects
ii. **Describing the system**: Class diagrams describe the attributes and operations of a class and also the constraints imposed on the system
iii. **Constructing executable code**: Class diagrams are used for constructing executable code of the software application

iv. **Mapping with object-oriented languages**: Class diagrams are the only UML diagrams that can be directly mapped with object-oriented languages, and thus they are widely used at the time of construction

v. **Preventing confusion**: Dealing with multiple objects in a system may be a source of challenges in learning the establishment of relationships between various attributes and objects in the system. Thus, a class diagram helps provide a particular visual representation to prevent unnecessary confusion.

OBJECT DIAGRAM

An object diagram is a UML structural diagram that shows the instances of the classifiers in models. It provides a snapshot of the system structure, capturing the static view of the instances present and their associations.

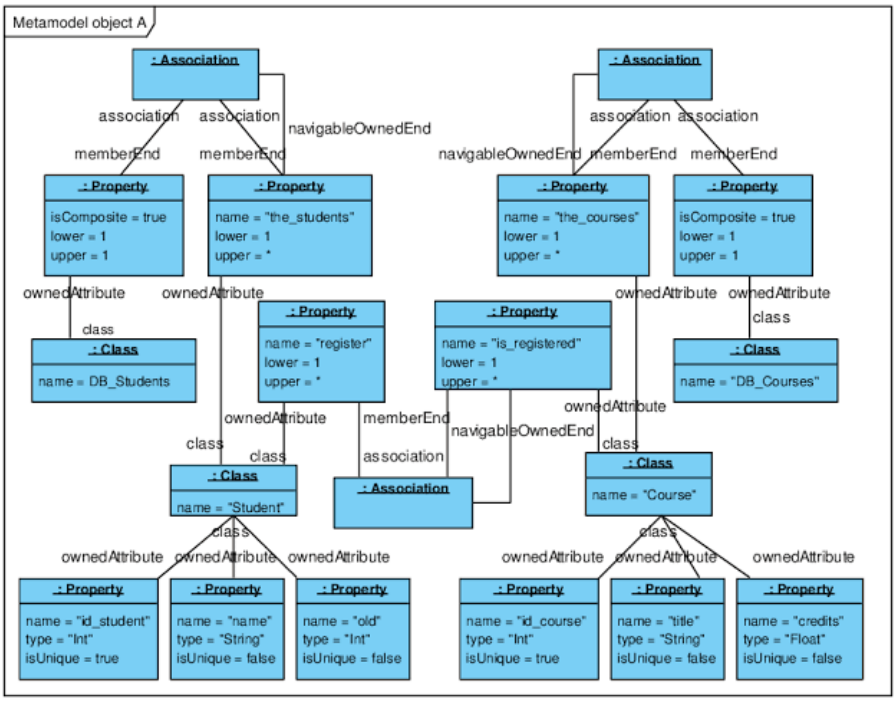

Fig 3. Object Diagram

USES OF OBJECT DIAGRAM

i. Object diagrams are UML structural diagrams that show the instances of classifiers in models.

ii. They provide a snapshot of the system at a particular moment, representing the static view of a system.

iii. Object diagrams are used to render a set of objects and their relationships as an instance, and they focus on the attributes of a set of objects and how those objects relate to each other.
iv. Object diagrams can be useful to explain smaller portions of a system when the system class diagram is very complex, and also sometimes modeling recursive relationships in a diagram.
v. They can also be used as a quick consistency check.
vi. To construct an object diagram, objects and links are the two elements used, and the object diagram should have a meaningful name to indicate its purpose

PROCESS OF MODELING WITH CLASS DIAGRAM

Class diagram is used to model the objects that make up the system, to display the relationships between the objects, and to describe what those objects do. A class diagram is made up of a description of a group of objects all with similar roles in the system, which consists of a class notation that consists of three parts: the graphical representation of the class, a list of attributes, and a list of operations. The graphical representation of the class is a rectangle with three compartments for the class name, attributes, and operations. The attributes and operations of a class are denoted by symbols such as +, -, and #, which denote the visibility of the attribute and operation. Class diagrams also show the relationships between classes, which can be one of the following types: inheritance (or generalization), simple association, and aggregation. Class diagrams are used throughout process modeling, which can be done with UML software

PROCESS OF MODELING WITH OBJECT DIAGRAM

An object diagram is a UML structural diagram that shows the instances of classifiers in models. It is similar to a class diagram, but it shows a snapshot of the system at a particular moment, representing the static view of a system. Object diagrams are used to render a set of objects and their relationships as an instance. They are useful to explain smaller portions of a system when the system class diagram is very complex, and also sometimes for modeling recursive relationships in a diagram

To create an object diagram, you can follow these steps:

- Analyze the system and decide which instances have important data and association.
- Consider only those instances that will cover the functionality.

- Make some optimization as the number of instances is unlimited.

Before drawing an object diagram, you should remember and understand the following:

- Object diagrams consist of objects.
- The link in an object diagram is used to connect objects.
- Objects and links are the two elements used to construct an object diagram.

IMPLEMENTATION DIAGRAM

Implementation diagrams are used to model the physical environment of a system and how its components will be deployed.

There are two main types of implementation diagrams:

i. Structural diagrams
ii. Behavioral diagrams

Structural diagrams: are used to describe the static structure of a system. They include the following types of diagrams:

- Class Diagram: This is the backbone of almost every object-oriented method, including UML. It describes the static structure of a system by showing the classes, their attributes and behaviors, and the relationships between each class
- Package Diagram: This diagram organizes elements of a system into related groups to minimize dependencies between packages.
- Object Diagram: This diagram shows a complete or partial view of the structure of a modeled system at a specific time.
- Component Diagram: This diagram represents a system decomposed into self-contained components or sub-systems. It shows the classifiers that make up these systems together with the artifacts that implement them, and exposes the interfaces offered or required by each component, and the dependencies between them.
- Composite Structure Diagram: This diagram shows the internal structure of a classifier, such as a class, and the collaborations that this structure makes possible.
- Deployment Diagram: This diagram shows the hardware on which each of the system components will run and how that hardware is connected together.

Behavioral diagrams: on the other hand, are used to describe the dynamic behavior of a system. They include the following types of diagrams:

- Activity Diagram: This diagram shows the flow from activity to activity within a system
- Use Case Diagram: This diagram gives a graphic overview of the actors involved in a system, different functions needed by those actors, and how these functions interact with the system
- Timing Diagram: This diagram shows the behavior of objects throughout a given period of time
- State Machine Diagram: This diagram shows the different states that an object can be in and how it transitions between those states
- Communication Diagram: This diagram shows the interactions between objects or parts in terms of sequenced messages
- Sequence Diagram: This diagram shows the interactions between objects or parts in terms of sequenced messages

COMPONENT DIAGRAM

A component diagram is a type of Unified Modeling Language (UML) diagram that depicts how components are wired together to form larger components or software systems. Component diagrams are essentially class diagrams that focus on a system's components and are used to model the static implementation view of a system.

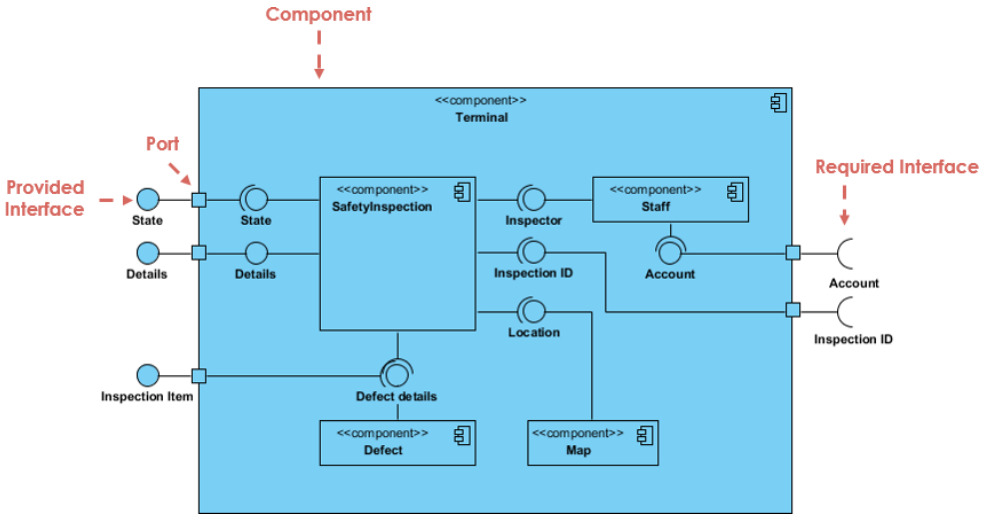

Fig 4. Component Diagram

USES OF COMPONENT DIAGRAM

Some of the uses of component diagrams include:

i. Visualizing the components of a system
ii. Describing the organization and relationships among components in a system
iii. Modeling the static implementation view of a system
iv. Helping to build executable systems through forward and reverse engineering
v. Checking that every aspect of the system's required functions is covered by planned development

DEPLOYMENT DIAGRAM

A deployment diagram is a type of structural diagram in the Unified Modeling Language (UML) that models the physical deployment of artifacts on nodes. It is used to describe the hardware components where software components are deployed. A deployment diagram shows the configuration of run time processing nodes and the components that live on them. It is used to visualize the topology of the physical components of a system, where the software components are deployed.

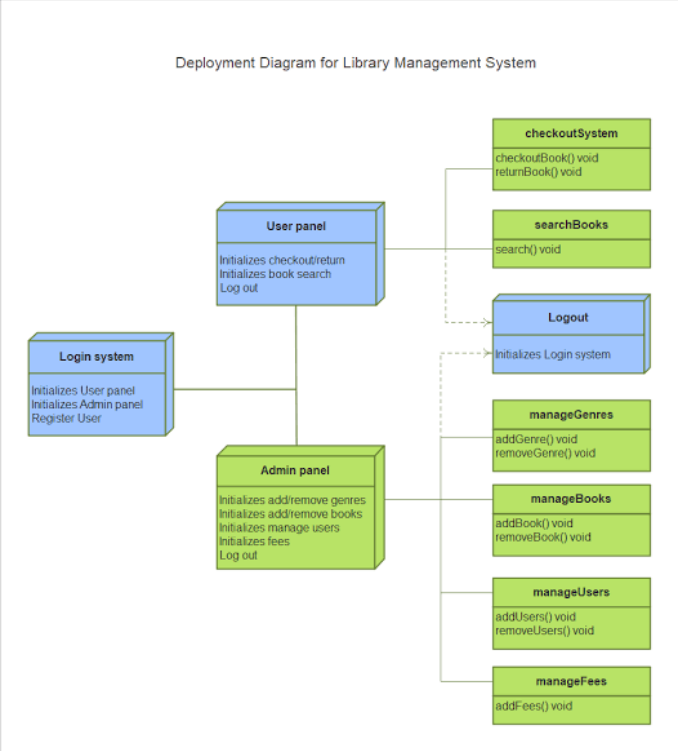

Fig. 5. Deployment Diagram

PURPOSE OF DEPLOYMENT DIAGRAM

i. To represent how software is istalled and interact with hardware component
ii. To visualize the topology of the physical component where software is deployed
iii. Deployment diagram important for system engineers as they help control parameters such as performance, scalability, maintainability, and portability

PROCESS OF MODELING WITH COMPONENT DIAGRAM

A component diagram is a UML diagram that visually represents how the components of a software system relate to one another. It is used to model the physical aspects of object-oriented systems and is used for visualizing, specifying, and documenting component-based systems. The diagram breaks down the actual system under development into various high levels of functionality, with each component responsible for one clear aim within the entire system and only interacting with other essential elements on a need-to-know basis.

To draw a component diagram, you can follow these steps:

- Figure out the purpose of the diagram and identify the artifacts such as the files, documents, etc. in your system or application that you need to represent in your diagram.
- As you figure out the relationships between the elements you identified earlier, create a mental layout of your component diagram.
- As you draw the diagram, add components first, grouping them within other components as you see fit.
- Next, add other elements such as interfaces, classes, objects, dependencies, etc. to your component diagram.

PROCESS OF MODELING WITH DEPLOYMENT DIAGRAM

Deployment diagrams are made up of several UML shapes, including nodes, which represent the basic software or hardware elements in the system, and lines that indicate relationships between nodes. The smaller shapes contained within the nodes represent the software artifacts that are deployed

To create a deployment diagram

- You should first identify the scope of your system and determine whether you are diagramming a single application or the deployment to a whole network of computers.
- You should also identify all nodes and the relationships between them before actually drawing the deployment diagram.
- When drawing the diagram, it is important to use clear and concise labels for each element and to ensure that the relationships between elements are clearly defined. Deployment diagrams are important for visualizing, specifying, and documenting embedded, client/server, and distributed systems, and for managing executable systems through forward and reverse engineering.
- They are typically used by software developers, system architects, and IT professionals who are responsible for designing and deploying software systems.

Deployment diagrams can be used to show which software elements are deployed by which hardware elements, illustrate the runtime processing for hardware, and provide a view of the hardware system's topology.

USE CASE DIAGRAM

A use case diagram is a graphical representation of the interactions between a user and a system.

It is a way to summarize the details of a system and the users within that system. Use cases specify the expected behavior of the system from the user's perspective, and not the exact method of making it happen.

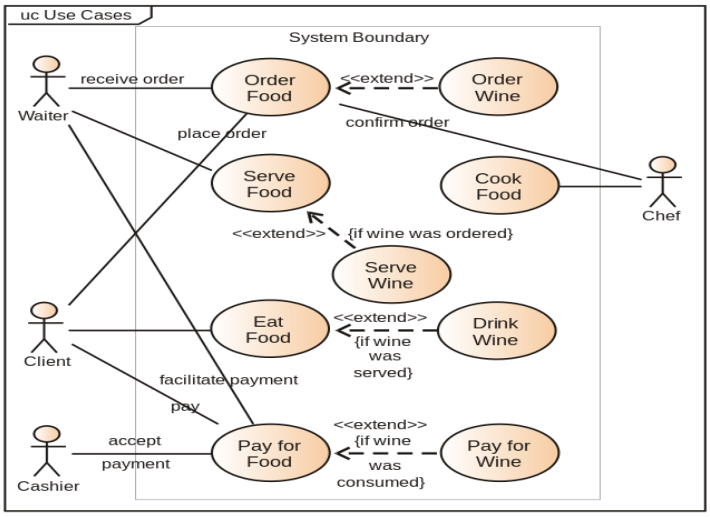

Fig. 6. Use Case diagram

The main components of a use case diagram include

i. The boundary
ii. Actors
iii. Use Cases
iv. The relationships between and among the actors and the use cases.

- The boundary defines the system of interest in relation to the world around it.
- The actors are individuals involved with the system defined according to their roles
- The use cases are the specific roles played by the actors within and around the system
- The relationships between and among the actors and the use cases are depicted using specialized symbols and connectors.

USES OF USE CASE DIAGRAM

The following are some of the uses of a use case diagram:

i. Represent the goals of systems and users
ii. Specify the context a system should be viewed in
iii. Specify system requirements
iv. Provide a model for the flow of events when it comes to user interactions
v. Provide an outside view of a system
vi. Show external and internal influences on a system

BASIC ELEMENTS AND NOTATIONS OF USE CASE DIAGRAM

The basic elements of a use case diagram are actors, use cases, and relationships.

Actors are external entities that interact with the system,

while use cases represent the distinct business functionalities of the system.

Relationships between actors and use cases are shown using connecting lines with optional arrowheads indicating the direction of control.

There are **three types** of relationships between actors and use cases:

Association, Generalization, and include.

TYPES OF USE CASE DIAGRAM

Use case diagrams are classified into two types:

i. Behavioral UML Diagrams
ii. Structural UML diagrams

Behavioral UML diagrams provide a visual representation of the dynamic behavior of a system, while structural UML diagrams show the static structure of a system.

Use case diagrams fall under the behavioral UML diagrams category and are used to demonstrate the different ways that a user might interact with a system. There are many different types of diagrams that can be used for designing and representing systems and processes, but use case diagrams are particularly useful for capturing requirements and expectations from a user's point of view.

USE CASE SPECIFICATIONS AND USE CASE TEMPLATE

A use case specification is a document that accompanies a use case diagram and explains the purpose of the use case and the functionality that is accomplished when the use case is executed. Use case templates and specifications are important because they provide a consistent format for documenting use cases, making it easier for stakeholders to understand and make decisions

A use case template typically includes the following sections:

- Use case name and identifier
- Actors involved in the use case
- Trigger that initiates the use case
- Preconditions that must be met before the use case can be executed
- Flow of events that describes the steps taken to accomplish the use case
- Alternate flows that describe different paths through the use case
- Postconditions that describe the state of the system after the use case is executed
- Priority level that indicates the importance of implementing the use case
- Assumptions made during the analysis of the use case

PROCESS OF MODELING WITH USE CASE

Use case summarizes the details of a system's users, also known as actors, and their interactions with the system.

Here are the steps to create a use case diagram:

- ✓ Identify the actors (roles of users) of the system.
- ✓ For each actor, identify the use cases (goals) that the actor can perform.

- ✓ Draw the use cases as ovals and the actors as stick figures.
- ✓ Connect the actors to the use cases with lines to show their participation in the system.
- ✓ Add include and extend relationships among use cases to show how they are related.
- ✓ Document the details of each use case using a use case template.
- ✓ Validate the actors and use cases with stakeholders to ensure accuracy, completeness, and consistency.

ACTIVITY DIAGRAM

An activity diagram is a type of flowchart that shows the flow of activities or actions in a system or process. It is a behavioral diagram in the Unified Modeling Language (UML) that captures the dynamic behavior of a system. Activity diagrams are used to model workflows of stepwise activities and actions with support for choice, iteration, and concurrency. They can describe the different dynamic aspects of a system, including the data flows intersecting with the related activities.. Activity diagrams are also used to model software architecture elements within a system by showing the relationships between the components and the constraints for assembling these components.

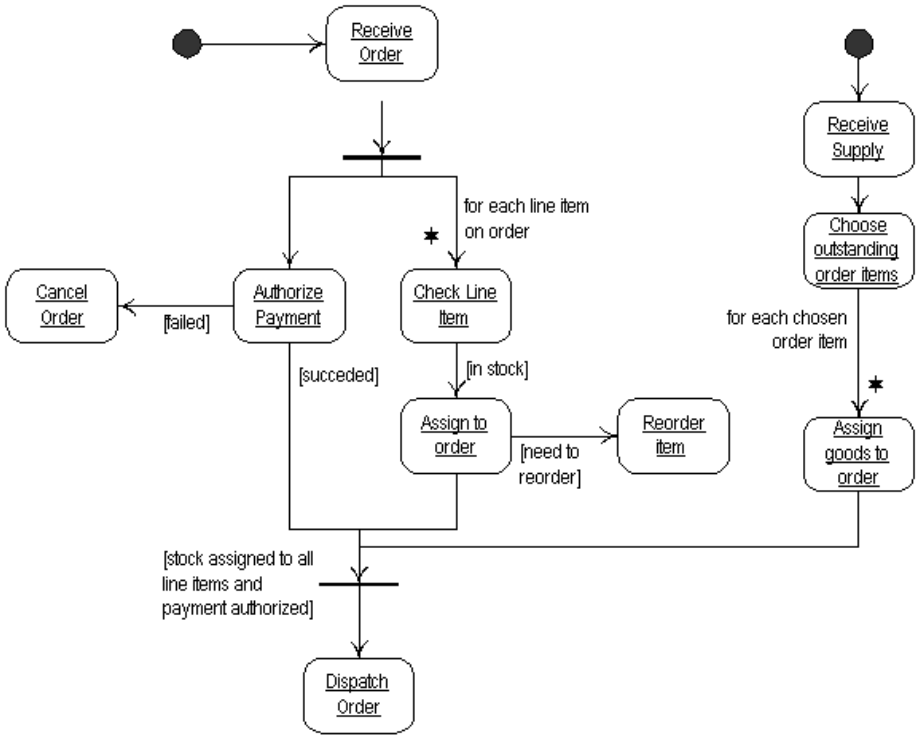

Fig. 7. Activity Diagram

USES OF ACTIVITY DIAGRAM

Activity diagrams are versatile tools that can be used in various fields to model workflows and processes. Here are some of the uses of activity diagrams:

i. Activity diagrams can be used to model workflows of stepwise activities and actions with support for choice, iteration, and concurrency. They can describe the different dynamic aspects of a system, including the data flows intersecting with the related activities.
ii. Activity diagrams are often used in business process modeling to show the progress of workflow between users and to identify bottlenecks within the system. They can also describe the steps in a use case diagram.
iii. Activity diagrams are used in software engineering to understand the flow of programs at a high level. They can be used to model software architecture elements within a system by showing the relationships between the components and the constraints for assembling these components
iv. Activity diagrams can be used to show customer journeys, such as the process of a user entering their credentials and logging in to a banking app
v. Activity diagrams provide a clear visualization of the logic of an algorithm, making it easy to identify what's working well and needs improvement.

BASIC ELEMENTS AND NOTATIONS OF ACTIVITY DIAGRAM

The basic elements and notations of an activity diagram include:

i. Initial state: The starting state before an activity takes place is depicted using the initial state. It is represented by a black filled circle.
ii. Final state: The state which the system reaches when a specific process end is known as a final state. It is represented by an encircled black circle.
iii. Action or activity box: It represents a step in the activity wherein the users or software perform a given task. It is symbolized with round-edged rectangles.
iv. Decision box: It is a diamond shape box that represents a decision with alternate paths. It includes a single input and two or more outputs.
v. Control flow arrow: It represents the order in which activities happen and connects the different elements of the activity diagram.
vi. Fork and join: They are represented by bars and are used to show the start and end of concurrent activities.

PROCESS OF MODELING WITH ACTIVITY DIAGRAM

To model a system or process using an activity diagram, you can follow these steps:

- ✓ Identify the activities: Identify the activities or actions that take place in the system or process. These activities can be sequential or concurrent.
- ✓ Determine the order of activities: Determine the order in which the activities take place. This can be done by drawing a flowchart of the activities.
- ✓ Add decision points: Add decision points to the flowchart to show where the flow of the system or process can change based on certain conditions.
- ✓ Add start and end points: Add start and end points to the flowchart to show where the system or process begins and ends.
- ✓ Add swim lanes: Add swim lanes to the flowchart to show which actors or groups are responsible for each activity.
- ✓ Add annotations: Add annotations to the flowchart to provide additional information about the activities or decision points.
- ✓ Validate the diagram: Validate the activity diagram with stakeholders to ensure accuracy, completeness, and consistency.

STATE CHART DIAGRAM

A state chart diagram, also known as a state diagram or state machine diagram, is used in computer science and related fields to describe the behavior of systems. It is a type of UML diagram that illustrates the various states of an object and the transitions between these states in response to events. State chart diagrams are especially useful for modeling the dynamic nature of a system, such as reactive systems that respond to external or internal events. They consist of states, transitions, events, and activities, and are used to represent the behavior of an object from creation to termination.

USES OF STATE CHART DIAGRAM

i. State chart diagram emphasizes the event-ordered behavior of an object and is especially useful in modeling reactive systems.
ii. State diagrams are used to model the states, transitions, events, and activities of a system component, such as a class, interface, or collaboration.
iii. They are beneficial for listing the events responsible for altering system states, modeling dynamic behavior and activity of a system.

iv. Demonstrating the response of a system to different types of stimuli, representing finite state machines visually, and illustrating the entire lifecycle of an object.

v. State chart diagrams are also used for forward and reverse engineering of a system, and their specific purpose is to define the state changes triggered by events

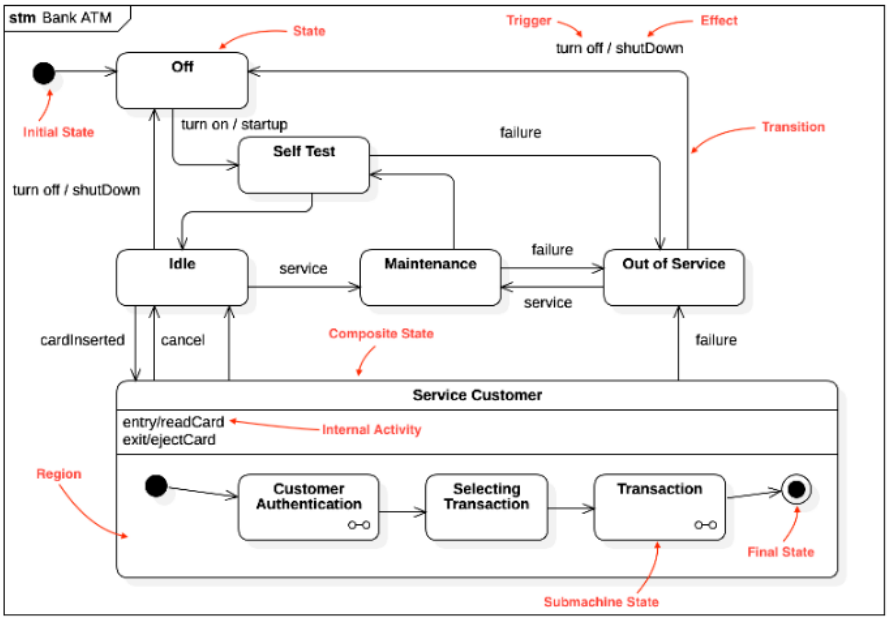

Fig. 8 State chart diagram

BASIC ELEMENTS OF STATE CHART DIAGRAM

- Transition: a transition refers to a change or shift from one state, subject, place, or situation to another.
- State Actions: are actions that occur when an object transitions from one state to another.
- Entry actions/point: These actions are defined for the target state and are invoked when the object enters the state. If the target state has many transitions arriving at it, and each transition has the same effect associated with it, it is better to associate the effect with the target state rather than the transitions
- Exit actions/point: These actions are defined for the source state and are invoked when the object leaves the state. Exit actions are the opposite of entry actions

- History States: A history state is used to remember the last substate that was active in a composite state before the transition from the composite state.
- Concurrent Regions: Concurrent regions, also known as orthogonal regions, allow multiple activities to occur simultaneously within the context of the enclosing object.

PROCESS OF MODELING WITH STATE CHART DIAGRAM

Modeling with a state chart diagram involves several steps and considerations. Here is a process of modeling using a state chart diagram:

- Identify the system or class: Determine the subject of the state chart diagram, which could be a system, class, or interface.
- Identify states and transitions: Determine the different states of the object and the events that trigger transitions between these states. States represent the conditions of the object, and transitions show how the object changes from one state to another
- Define events: Events are external or internal stimuli that cause the object to change states. They act as triggers for state transitions and are represented by bars or arrows in the diagram
- Draw the state chart diagram: Create a visual representation of the states, transitions, and events using the UML notation. State chart diagrams use rectangles for states, arrows for transitions, and bars or arrows for events
- Ensure uniqueness and clarity: Make sure the names of states and transitions are unique and easily understandable. This will help in maintaining the clarity and readability of the diagram
- Analyze and refine: Review the state chart diagram to ensure it accurately represents the behavior of the object or system. Make any necessary adjustments to improve the clarity and accuracy of the diagram

INTERACTION DIAGRAM

An interaction diagram is a type of UML diagram that captures the interactive behavior of a system. It focuses on describing the flow of messages within a system, providing context for one or more lifelines within a system. Interaction diagrams can be used to represent the ordered sequences within a system and act as a means of visualizing real-time data via UML. They can be implemented in a number of scenarios to provide a unique set of information, such as modeling a system as a time-ordered sequence of events, reverse- or forward-engineering a system or

process, organizing the structure of various interactive events, simply conveying the behavior of messages and lifelines within a system, and identifying possible connections between lifeline elements.

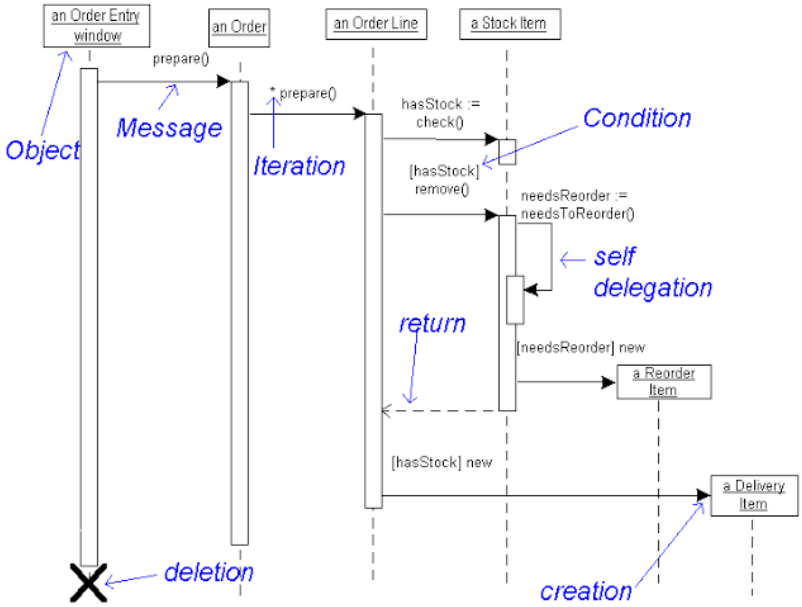

Fig. 9. Interaction Diagram

TYPES OF INTERACTION DIAGRAM

There are four types of interaction diagrams in UML, which are:

i. Sequence diagram: It emphasizes the time sequence of messages and shows the order in which messages are exchanged between objects
ii. Collaboration diagram: It emphasizes the structural organization of the objects that send and receive messages and shows the relationships between objects
iii. Timing diagram: It shows how objects change state over time and how long they stay in each state
iv. Interaction overview diagram: It provides an overview of the flow of control of the interactions and shows how a set of fragments might be initiated in various scenarios.

SEQUENCE DIAGRAM

A sequence diagram is a type of interaction diagram in UML that shows the interactions between objects in a system or classes within code. It emphasizes the time sequence of messages and shows the order in which messages are exchanged between objects. Sequence diagrams are used to model use case scenarios, including

main success scenarios and alternate scenarios. They are organized according to time, with the time progressing as you go down the page, and the objects involved in the operation listed from left to right according to when they take part in the message sequence.

USES OF SEQUENCE DIAGRAM

Some of the uses of sequence diagrams are:

i. Modeling system behavior: Sequence diagrams are used to model the behavior of a system at a high level of abstraction. They show the interactions between objects in a system and the order in which these interactions take place
ii. Documenting requirements: Sequence diagrams can be used to document the requirements for a new system or to document an existing process. They provide a clear and concise visualization of the interactions between objects in a system
iii. Designing new systems: Sequence diagrams can be used to design new systems by showcasing the system's object interactions and putting out a more fleshed-out overall system design
iv. Reverse engineering: Sequence diagrams can be used to reverse-engineer a system or process by documenting how objects in an existing system currently interact
v. Communicating with stakeholders: Sequence diagrams can be used to communicate with stakeholders, including business staff, technical staff, and software developers, to showcase how the system works and to understand the requirements for a new system

ELEMENT AND NOTATIONS OF SEQUENCE DIAGRAM

The basic elements and notations of a sequence diagram include:

- ✓ Lifeline: A vertical dashed line that represents an object or participant in the interaction.
- ✓ Message: An arrow that represents a communication between two lifelines. It can be synchronous, asynchronous, or a return message.
- ✓ Activation bar: A horizontal bar that represents the time during which an object is performing an action.
- ✓ Combined fragment: A box that groups messages together and represents a condition or loop.

- ✓ Interaction use: A box that represents a reference to another sequence diagram.
- ✓ Object destruction: A cross that represents the destruction of an object.
- ✓ Self-message: A message that loops back to the same lifeline.

COLLABORATION DIAGRAM

A collaboration diagram, also known as a communication diagram, is a type of UML diagram that shows the relationships and interactions among software objects or classes within code. It emphasizes the structural organization of the objects that send and receive messages and shows the relationships between objects. Unlike a sequence diagram, a collaboration diagram does not show time as a separate dimension, so sequence numbers determine the sequence of messages and the concurrent threads.

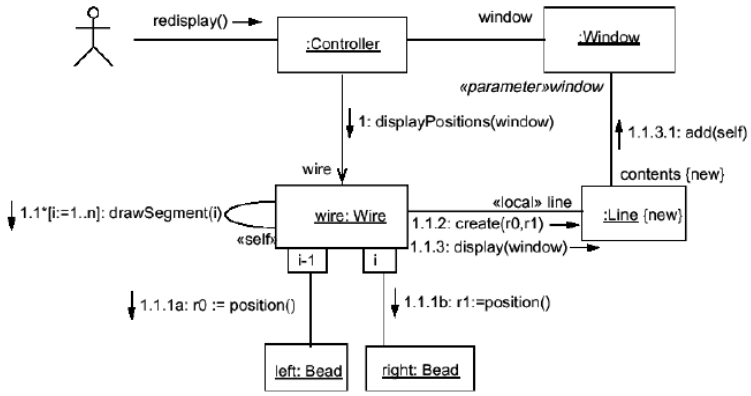

Fig. 10. Collaboration Diagram

USES OF COLLABORATION DIAGRAM

Some of the uses of collaboration diagrams are:

i. Modeling system behavior: Collaboration diagrams are used to model the behavior of a system by showing the relationships and interactions among software objects or classes within code

ii. Documenting requirements: Collaboration diagrams can be used to document the requirements for a new system or to document an existing process. They provide a clear and concise visualization of the relationships and interactions among software objects or classes within code.

iii. Designing new systems: Collaboration diagrams can be used to design new systems by showcasing the relationships and interactions among software objects or classes within code
iv. Reverse engineering: Collaboration diagrams can be used to reverse-engineer a system or process by documenting the relationships and interactions among software objects or classes within code
v. Communicating with stakeholders: Collaboration diagrams can be used to communicate with stakeholders, including business staff, technical staff, and software developers, to showcase how the system works and to understand the requirements for a new system.

PROCESS OF MODELING WITH COLLABORATION DIAGRAM

To create a collaboration diagram, you can follow these steps:

- Identify the objects: Identify the objects or classes within code that are involved in the interaction.
- Determine the relationships: Determine the relationships between the objects or classes within code. This can be done by drawing a flowchart of the relationships.
- Add messages: Add messages to the flowchart to show the interactions between the objects or classes within code.
- Add notations: Add notations to the flowchart to provide additional information about the objects or classes within code.
- Validate the diagram: Validate the collaboration diagram with stakeholders to ensure accuracy, completeness, and consistency.

SYSTEM MODEL CONVERSION

System model conversion refers to the process of changing from an old system to a new one. There are different approaches to system conversion, including direct conversion, parallel conversion, modular conversion, and phase-in conversion. The choice of approach depends on the specific requirements of the system and the organization's needs. Collaboration diagrams and sequence diagrams are two types of UML diagrams that can be used to model system behavior and interactions between objects or classes within code.

Model conversion can also refer to the process of converting a model from one representation to another. Conversion modeling is a process that helps organizations

understand how their customers interact with their products or services and optimize their marketing efforts to better convert leads and customers.

IMPORTANCE OF MODEL CONVERSION

Model conversion is important for various reasons, depending on the context. Here are some examples.

- Actuarial model conversion: Actuarial model conversion can help advance an actuary practice by providing an opportunity to improve their models and processes.
- Digital marketing: Conversion models are essential in digital marketing, as they help advertisers understand the factors that influence user behavior and optimize their campaigns to drive better results such as sales, registrations, and subscriptions.
- Conversion modeling involves tracking and evaluating user interactions with marketing campaigns, websites, and mobile applications to identify the most effective strategies for driving conversions
- Understanding customer behavior: Conversion modeling helps organizations understand how their customers interact with their products or services and optimize their marketing efforts to better convert leads and customers. It helps them improve their processes and increase their sales and profits
- Machine learning: Conversion modeling refers to the use of machine learning to quantify the impact of marketing efforts when a subset of conversions can't be observed. It can help accurately assess the effectiveness of marketing investments and provide a complete view of performance

PROCESS OF CONVERTING UML DIAGRAMS INTO PROGRAM CODE

To convert UML diagrams into program code, you can use a code generator tool that allows you to convert your diagrams to source code in various programming languages. The code generator supports various programming languages, including Java, C#, Python, C++, and others.

The process of converting UML diagrams into program code involves the following steps:

- Create a UML diagram: Create a UML diagram using a UML modeling tool.

- Generate code: Use a code generator tool to generate code from the UML diagram. The code generator tool will convert the UML diagram into source code in the programming language of your choice.
- Refine the code: Refine the generated code to ensure that it meets the requirements of the system.
- Compile and test the code: Compile and test the code to ensure that it works as expected.

PROCESS OF CONVERTING PROGRAM CODE INTO UML DIAGRAMS

The process of converting program code into UML diagrams can be challenging, but there are tools and techniques available to help with this process. Here are some steps that can be followed to convert program code into UML diagrams:

- Analyze the code: Analyze the program code to understand the structure and behavior of the system.
- Identify the classes and objects: Identify the classes and objects within the code that are involved in the system.
- Create a class diagram: Create a class diagram that represents the classes and objects within the code and their relationships.
- Add attributes and methods: Add attributes and methods to the class diagram to represent the behavior of the system.
- Validate the diagram: Validate the class diagram with stakeholders to ensure accuracy, completeness, and consistency.

References

http://www.uml.org/

Sparx Systems

http://www.sparxsystems.com/resources/uml2_t

utorial/index.html

Learners support publication

http://www.lsp4you.com/seminar.htm